

CoGaussian: Open-Vocabulary Gaussian-based Collaborative 3D Scene Understanding

Anonymous Authors

Abstract— Collaborative perception enables agents to retrieve the scene semantics and geometry beyond the field of view of any individual agent. This broader scene understanding enhances transportation safety and improves efficiency in multi-robot systems. Existing collaborative semantic occupancy methods, however, require dense 3D voxel annotations and have a fixed vocabulary. To overcome these limitations, we present CoGaussian, an open-vocabulary Gaussian-based collaborative semantic occupancy prediction framework. CoGaussian represents a scene as a set of foundation model-aligned sparse 3D Gaussians, and fuses geometry and semantic features across multiple agents through three methods: zero-shot concatenation, joint training, and radius-gated attention. Our method outperforms existing open-vocabulary single-agent approaches on mIoU and occupancy IoU, highlighting the effectiveness of open-vocabulary collaborative scene understanding.

I. INTRODUCTION

Collaborative perception allows nearby agents to exchange information that is occluded or outside their field-of-view, enhancing reasoning about surrounding geometry and semantics and enabling safer and more aware movements. A common target for 3D spatial understanding is Semantic Occupancy Prediction (SOP), which aims to predict voxel-wise occupancy status and semantic class. Recently, 3D Gaussian-based methods have been shown to be effective for SOP, highlighted by the GaussianFormer [1], [2], [3] family, which utilize dense voxel-based 3D supervision for training. VOGS-CP [4] has extended GaussianFormer to collaborative SOP which communicates Gaussian primitives between neighboring agents. However, these methods support a closed vocabulary, and rely on densely annotated 3D occupancy labels that are expensive to retrieve.

Foundation models such as DINOv2 [5], CLIP [6], and Talk2DINO [7] provide a way to avoid expensive annotation, making open-vocabulary scene understanding practical. Methods such as GaussTR [8] represent scenes with a sparse set of 3D Gaussians that have been predicted in a feed-forward manner, learning geometry and semantics by splatting the Gaussians to 2D views and aligning rendered features with foundation model-derived depth and semantics. However, GaussTR is designed for single-agent perception, and thus can not retrieve information visible only to neighboring agents.

To address these limitations, we propose the first open-vocabulary multi-robot collaborative perception framework named **CoGaussian**. We use sparse VFM-aligned 3D Gaussians for scene representation, which also serve as the communication primitive among agents. We also develop and study three methods to aggregate information from nearby agents. We summarize our contributions as follows:

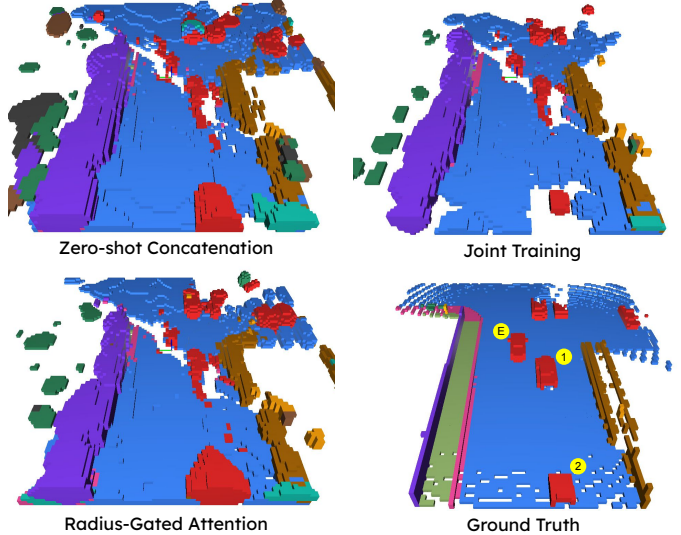


Fig. 1. **Comparison of three variants of CoGaussian with Ground Truth.** Visualization of the experiments in Sec. III. The ego vehicle is marked as E. Another agent is in the front-right (not visible here), partially completing the front-right of the ego vehicle in collaborative predictions. Class legends are shown in Table I.

- **Open-Vocabulary Collaborative Perception:** We propose CoGaussian, the first framework for open-vocabulary collaborative perception in outdoor environments for 3D Semantic Occupancy Prediction (SOP).
- **Multi-Robot 3D Gaussian Representation:** We develop a foundation model-aligned 3D Gaussian representation for collaborative perception, along with fusion strategies to enhance information transfer among agents.
- **Zero-Shot Benchmarking:** We evaluate CoGaussian on the Semantic-OPV2V [9], [10] dataset under a *zero-shot* setting. Without any task-specific supervision, our approach achieves impressive performance on semantic occupancy prediction, highlighting the potential of open-vocabulary collaborative perception.

II. METHOD

We represent a scene as a sparse set of 3D Gaussians with associated DINOv2 features. Predicted Gaussians are splatted back into the source camera and supervised by DINOv2 features and Metric3Dv2 monocular depths [11]. During inference, semantic class logits are computed as similarity between predicted Gaussian features and Talk2DINO text embeddings for each class [7].

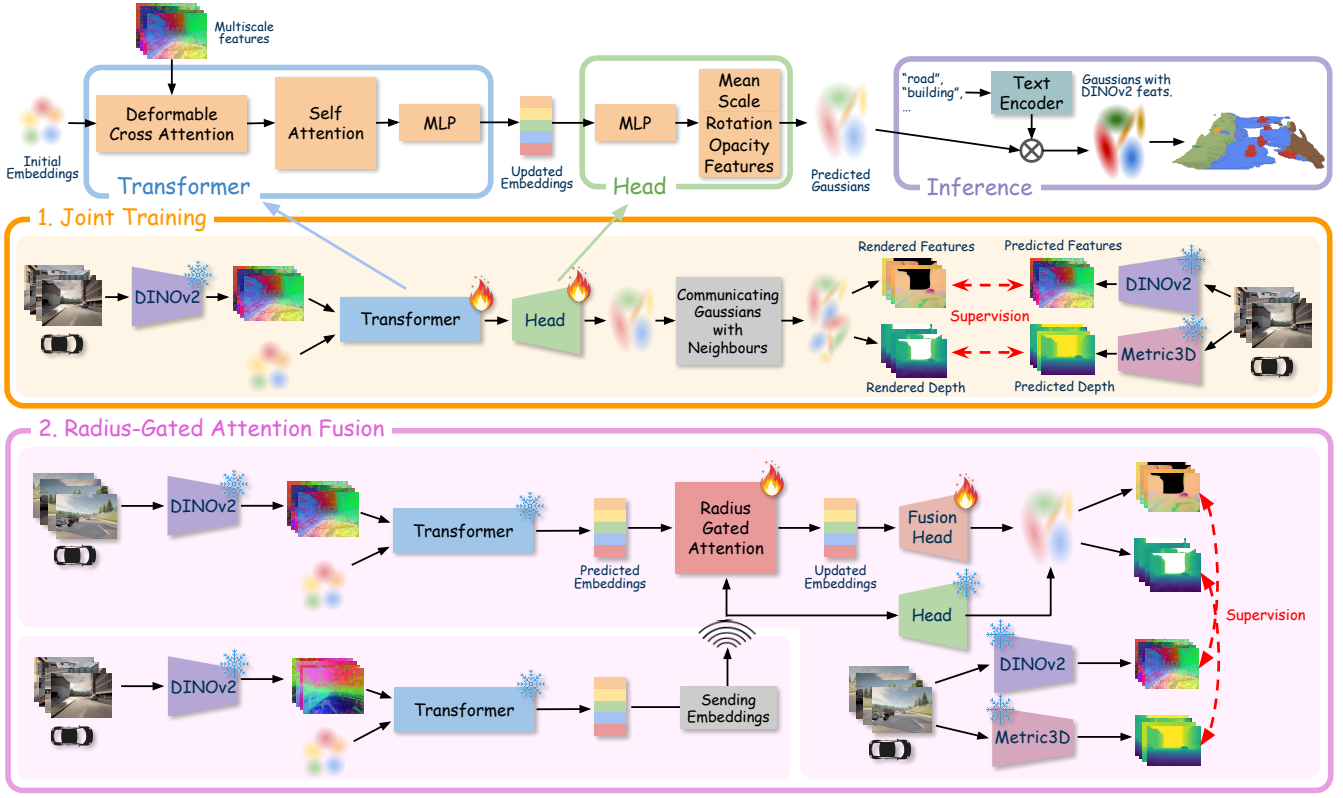


Fig. 2. **Overview.** CoGaussian starts with randomly initialized embeddings. A transformer layer uses multiscale features of images to update the embeddings, followed by a Head Layer that predicts Gaussians. During Joint Training, all agents predict *ego Gaussians* which are communicated with each other and aggregated. This set of gaussians is used for training. In the Fusion pipeline, the neighbors predict and send Gaussian embeddings to the ego agent. Radius-gated Attention uses these *neighbor Gaussian* embeddings to update ego embeddings. The Fusion Head predicts ego Gaussians, and pre-trained Head from Joint Training predicts neighbor Gaussians. These are aggregated to form the final prediction used for both training and inference.

A. Preliminaries

Each 3D Gaussian G is parametrized as:

$$G = \{\boldsymbol{\mu} \in \mathbb{R}^3, \mathbf{s} \in \mathbb{R}^3, \mathbf{r} \in \mathbb{R}^4, \alpha \in [0, 1], \mathbf{f} \in \mathbb{R}^D\}, \quad (1)$$

where $\boldsymbol{\mu}$ is the mean (center), $\mathbf{s}$ is the per-axis scale, $\mathbf{r}$ the rotation quaternion of the axes, α is the opacity, and $\mathbf{f}$ is the DINOv2 feature vector.

The model inputs a single image and initializes a set of Gaussian embeddings $\{\mathbf{q}_i \in \mathbb{R}^E\}_{i=1}^N$, where E denotes the embedding dimension. Each embedding is associated with a randomly sampled image location $(u_i, v_i) \in [0, 1]^2$ and the corresponding Metric3D depth obtained at that location. These are projected into 3D space using camera intrinsics and extrinsics to represent the Gaussian's mean $\boldsymbol{\mu}$.

Once the Gaussian embeddings are initialized, they are refined through a transformer layer $\mathcal{F}$ that consists of a deformable attention [12], a self-attention, and an MLP layer, as shown in Fig. 2. The point (u_i, v_i) serves as the reference point for deformable attention over the image features:

$$\mathbf{q}'_i = \mathcal{F}(\mathbf{q}_i, (u_i, v_i)). \quad (2)$$

Finally, the Gaussian parameters are predicted from embeddings $\mathbf{q}_i$ using MLPs within a Gaussian Head:

$$g_i := \{\Delta\boldsymbol{\mu}_i, \mathbf{s}_i, \mathbf{r}_i, \alpha_i, \mathbf{f}_i\} = \text{MLP}_{\text{GH}}(\mathbf{q}'_i). \quad (3)$$

The means $\boldsymbol{\mu}$ are iteratively refined after each layer using the $\Delta\boldsymbol{\mu}$. The scale $\mathbf{s}$ and opacity α are scaled to appropriate ranges. The rotation $\mathbf{r}$ is axis-aligned in the camera's coordinate frame. It is not learned to simplify training.

The model is trained by rendering the DINOv2 features and the depth using differentiable alpha-blending over all Gaussians employing optimised CUDA kernels [13]. The loss terms penalize the difference between rendered depth and features, and predicted depth and features [8].

B. Joint Training

GaussTR is trained per image, i.e. Gaussians predicted from one camera are splatted back into that camera for supervision. It works well for cameras with little or no overlap in field of views, such as in the nuScenes dataset [14]. However, in a multi-agent setting, training the model for each camera separately completely disregards that different agents observe overlapping regions.

Therefore, in CoGaussian the model is trained *per scene* as shown in Fig. 2. All the Gaussians predicted from all the cameras in a scene are aggregated into a common coordinate frame. And this set of Gaussians is splatted into every camera in which they are visible, not only in the cameras that produced them. So, predictions consistent with a single view but inconsistent with other views are penalized.

During inference, Gaussians are predicted from each camera, aggregated, and used by agents for downstream tasks. Note that CoGaussian could utilise weights from both the model trained per image and the model trained per scene. In Sec. III, we call the former *Zero-shot concatenation*, i.e. uses per image training with our proposed inference method, and the later *Joint Training*, which instead uses training per scene to enable collaboration.

C. Radius-Gated Attention Fusion

Joint Training is shown to improve performance in Table I. However, aggregating Gaussians from different cameras, with overlapping field of views causes them to overlap. While this adds additional information for the ego, it also introduces more noise. Additionally, the appearance of the same object often varies from different viewpoints. Thus, Gaussians for the same object may differ widely. Joint training contributes in mitigating such differences, but it handles this *implicitly* along with predicting Gaussians. So, instead, we make this *explicit* by introducing a *fusion* module.

The fusion module is responsible for aggregating useful information from Gaussians predicted by neighboring agents. We use a single-head scaled dot product attention [15] between the two Gaussian sets: ego Gaussian embeddings and neighboring Gaussian embeddings. Afterwards, a *Fusion Head* of MLPs with the same architecture as the Gaussian Head is used to predict Gaussians from the updated embeddings. The updated ego Gaussians, as well as the neighboring Gaussians are aggregated to form the final set that is used for downstream tasks. The fusion pipeline is shown in Fig. 2.

Selecting neighbor Gaussians. Let $\mathcal{G}_{ego}$ be the set of ego Gaussians and $\mathcal{G}_{ngh}$ be the set of neighbor Gaussians, transformed into ego's coordinate frame. For each ego gaussian $g_i \in \mathcal{G}_{ego}$, we take its k nearest neighbors in $\mathcal{G}_{ngh}$ by Euclidean distance between means of Gaussians, denoted by $\mathcal{N}(g_i)$. Then, we create an attention mask $\mathbf{M}$ based on distance between g_i and $g_j \in \mathcal{N}(g_i)$:

$$M_{i,j} = \mathbb{1}[\|\mu_i - \mu_j\|_2 \leq \rho]. \quad (4)$$

Both k and ρ are hyperparameters. Taking the k nearest neighbors imposes an intuitive locality bias: an ego gaussian can only be updated by nearby neighbor Gaussians. The bias is more strongly enforced by allowing Gaussians only within a specific radius ρ to contribute to the update.

Attention over Gaussian embeddings. The fusion uses Gaussian embeddings obtained from the transformer layers on both ego and neighboring agents along with the optimised mean. The neighbor Gaussian embeddings and their means are communicated to the ego agent, where the attention matrix is computed:

$$\Delta\mu_{i,j} = \mu_j - \mu_i, \quad \mathbf{z}_{i,j} = [\mathbf{q}_j \parallel \Delta\mu_{i,j} \parallel \eta(\Delta\mu_{i,j})], \quad (5)$$

$$\mathbf{Q}_i = \phi_q(\mathbf{q}_i), \quad \mathbf{K}_{i,j} = \phi_k(\mathbf{z}_{i,j}), \quad \mathbf{V}_{i,j} = \phi_v(\mathbf{z}_{i,j}), \quad (6)$$

$$A_{i,j} = \frac{\langle \mathbf{Q}_i, \mathbf{K}_{i,j} \rangle}{\sqrt{d}}. \quad (7)$$

Here, $\mathbf{q}_i$ is the embedding corresponding to Gaussian g_i . $\Delta\mu_{i,j}$ is the relative position of g_j with respect to g_i . ϕ_q , ϕ_k , and ϕ_v are two-layer MLPs and $\parallel$ is concatenation. The query vector is built from ego Gaussian embedding alone. The key and value vectors combine the neighbor Gaussian embeddings with its position relative to ego Gaussian along with its positional encoding $\eta(\Delta\mu_{i,j})$. d is the dimension of $\mathbf{Q}_i$ and $\mathbf{K}_{i,j}$.

$$w_{i,j} = \frac{\text{softmax}_j(A_{i,j}) M_{i,j}}{\sum_{j'} \text{softmax}_{j'}(A_{i,j'}) M_{i,j'} + \epsilon}, \quad (8)$$

$$\mathbf{c}_i = \sum_{j \in \mathcal{N}(g_i)} w_{i,j} \mathbf{V}_{i,j}, \quad \hat{\mathbf{q}}_i = \mathbf{q}_i + \lambda \psi(\mathbf{c}_i). \quad (9)$$

The attention mask is applied while calculating the softmax, which allows only radius-gated neighboring Gaussians to have non-zero weights $w_{i,j}$. An MLP ψ is used to obtain an update which is scaled by λ and added to the Gaussian embedding.

Training. We follow a two-stage training paradigm. Stage 1: We train the Transformer Layer and Gaussian Head as described in Sec. II-B. Stage 2: The Transformer Layer and Gaussian Head are frozen, and the model learns the Radius-Gated Attention Layer and the Fusion Head.

III. EXPERIMENTS

A. Dataset and Metrics

We evaluate on the Semantic-OPV2V dataset [10], an extension of OPV2V dataset [9] with 3D SOP ground truth. OPV2V is a simulated perception benchmark for vehicle-to-vehicle (V2V) collaboration with two to seven collaborating agents in a scene (Not all visible vehicles collect sensor data). Each vehicle is equipped with 4 cameras. The ground truth consists of 12 semantic labels over a $40 \times 40 \times 3.2m$ area with a grid size of $100 \times 100 \times 8$, giving a voxel size of $0.4m$.

Following previous work [4], [10], we report the per-class Intersection-over-Union (IoU), and its mean (mIoU), along with the overall occupancy IoU for our CoGaussian implementation. Even though fully-supervised methods are included for reference, they are not comparable to our approach as they use dense 3D voxel supervision.

B. Implementation Details

CoGaussian was trained for 3 epochs, taking approximately 14 hours on a single NVIDIA A100 GPU. The losses converged after 2 training epochs. Metric3Dv2 [11] provides depth supervision, and DINOv2 provides open-vocabulary feature supervision. At inference, prior to voxelization, Gaussian features are compared with text embeddings that have been aligned to DINOv2 with Talk2DINO. As a result, we prompt Talk2DINO over all 12 Semantic-OPV2V classes plus an extra class to represent free space. We use AdamW at a learning rate of 1×10^{-5} with weight decay of 5×10^{-3} , and gradient clipping of 20. We evaluate models using 1200 Gaussian queries per camera in 4 configurations:

TABLE I
3D SEMANTIC OCCUPANCY PREDICTION ON SEMANTIC-OPV2V DATASET

Method	Fully-Supervised, Closed Vocabulary						Self-Supervised, Open Vocabulary			
	Single-Agent		Collaborative Perception				Single-Agent	Collaborative Perception		
	CoHFF	GSFormer	CoHFF	VOGS-CP			GaussTR	CoGaussian (Ours)		
				Zero Shot	Naive Fusion	Learned Fusion		Concatenation	Trained	Attention
IoU ($\uparrow$)	38.52	67.76	50.46	67.88	<u>70.10</u>	72.87	20.58	16.13	<u>22.54</u>	22.57
mIoU ($\uparrow$)	24.81	29.20	34.16	30.54	<u>36.02</u>	37.44	6.26	4.81	<u>6.74</u>	6.95
Building 	21.04	3.84	25.72	3.91	7.18	<u>9.61</u>	2.03	1.25	1.65	<u>1.93</u>
Fence 	20.05	14.10	<u>27.83</u>	17.29	25.66	29.20	3.19	2.61	3.80	<u>3.39</u>
Terrain 	43.93	68.97	48.30	72.00	76.05	<u>74.51</u>	<u>2.86</u>	1.71	2.27	2.97
Pole 	31.66	5.94	42.74	8.44	12.67	12.19	0.95	0.87	<u>0.96</u>	1.04
Road 	55.83	79.37	61.77	<u>81.35</u>	78.60	83.05	27.68	19.71	<u>32.41</u>	32.70
Side walk 	17.31	70.55	39.62	68.91	75.88	78.22	<u>14.90</u>	12.57	13.62	17.29
Vegetation 	14.49	12.54	20.59	13.72	16.42	<u>20.43</u>	<u>5.67</u>	4.64	5.88	5.39
Vehicles 	58.55	49.25	63.28	49.56	57.14	<u>60.49</u>	12.55	9.88	15.39	<u>14.24</u>
Wall 	33.30	30.79	58.27	30.49	35.25	<u>36.45</u>	3.17	<u>2.83</u>	2.49	2.53
Guard rail 	1.54	15.01	1.94	20.78	41.00	<u>32.50</u>	<u>0.31</u>	0.16	0.85	<u>0.31</u>
Traffic signs 	0	0	16.33	0	6.35	<u>8.26</u>	<u>0.45</u>	0.44	0.52	0.52
Bridge 	0	0	<u>3.53</u>	0	0	4.35	1.33	<u>1.08</u>	1.07	1.05

- **Single Agent Perception:** Each agent independently predicts occupancy.
- **Zero-shot Concatenation:** Gaussians predicted by all agents are aggregated in a common coordinate frame.
- **Joint Training:** All the aggregated Gaussians from all cameras are used for fine-tuning.
- **Radius-Gated Attention:** The fusion pipeline runs in all agents updating ego Gaussians using neighbor Gaussians.

IV. RESULTS AND DISCUSSION

A. Quantitative Results

Table 1 highlights quantitative results from previous works, as well as our experiments. Relative to the single-agent GaussTR baseline, zero-shot concatenation reduces performance by $-4.45/-1.45$ (IoU/mIoU), showing that naive Gaussian aggregation introduces spatially inconsistent structure. Joint Training improves from the baseline by $+1.96/+0.48$ (IoU/mIoU), indicating that cross-view rendering supervision reconciles these inconsistencies. Using Radius-Gated Attention gives a modest gain to $+1.99/+0.69$ (IoU/mIoU) over the baseline.

Class-wise behaviour is mixed, with frequently observed classes like "road", "sidewalk", and "vehicles" benefiting the most from collaboration, as they have major overlap among agents and thus a stronger supervision signal. Prominent but less visible classes like "vegetation" and "terrain" struggle with collaboration and other small and uncommon classes have mixed results.

B. Qualitative Results

Fig. 1 shows that the collaborative methods recover structure outside the direct view of the ego, most notably the ego vehicle itself and the front-right region. Our method tends to produce volumetric predictions instead of one voxel thick surfaces visible in the ground truth. It marks the distinctive

feature of sparse 3D Gaussian representation as opposed to methods like [1], [2], [3] that rely on 3D semantic voxel supervision with a larger number of small Gaussians representing surfaces in the scene. 2D image-based supervision causes objects taking up a large area in the image to be prominent in 3D space. The elongated prediction of Vehicle 1 and voluminous prediction of Vehicle 2 demonstrate these effects.

Joint Training reduces these inconsistencies, producing smaller and more conservative Gaussians, evident in Vehicle 2 prediction and comparatively smaller elliptical predictions throughout the scene. The fusion pipeline preserves this coherence while recovering more of the dominant parts of the scene like the Road and some Vehicles. However, thin and infrequent structures still remain difficult to reconstruct reliably under purely 2D self-supervision.

V. CONCLUSION AND FUTURE WORK

We introduce CoGaussian which is, to the best of our knowledge, the first framework for open-vocabulary collaborative perception that communicates foundation model-aligned 3D Gaussians among agents representing a scene. Evaluation shows that collaboration works when interaction is learned. Zero-shot collaboration hurts performance, whereas joint training and radius-gated attention improve over single-agent baseline, giving promising results. All the while, it brings the flexibility of open-vocabulary tasks and integration to collaborative perception.

We plan to further experiment with different fusion architectures, and evaluate on other downstream tasks such as BEV Semantic Segmentation and 3D Object Detection. An efficient communication paradigm along with inference optimisation is also crucial for enabling real-time implementation, bringing open-vocabulary to enable diverse tasks.

REFERENCES

- [1] Y. Huang, W. Zheng, Y. Zhang, J. Zhou, and J. Lu, "GaussianFormer: Scene as Gaussians for Vision-based 3D Semantic Occupancy Prediction," in *ECCV*, 2024.
- [2] Y. Huang, A. Thammadatrakoon, W. Zheng, Y. Zhang, D. Du, and J. Lu, "GaussianFormer-2: Probabilistic Gaussian Superposition for Efficient 3D Occupancy Prediction," in *CVPR*, 2025.
- [3] L. Zhao, S. Wei, J. Hays, and L. Gan, "Gaussianformer3d: Multi-modal gaussian-based semantic occupancy prediction with 3d deformable attention," *arXiv preprint arXiv:2505.10685*, 2025.
- [4] C. Chen, H. Huang, and S. Bagchi, "Vision-only gaussian splatting for collaborative semantic occupancy prediction," in *AAAI*, 2026.
- [5] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, R. Howes, P.-Y. Huang, H. Xu, V. Sharma, S.-W. Li, W. Galuba, M. Rabbat, M. Assran, N. Ballas, G. Synnaeve, I. Misra, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "Dinov2: Learning robust visual features without supervision," in *TMLR*, 2024.
- [6] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *ICML*, 2021.
- [7] L. Barsellotti, L. Bianchi, N. Messina, F. Carrara, M. Cornia, L. Baraldi, F. Falchi, and R. Cucchiara, "Talking to dino: Bridging self-supervised vision backbones with language for open-vocabulary segmentation," in *ICCV*, 2025.
- [8] H. Jiang, L. Liu, T. Cheng, X. Wang, T. Lin, Z. Su, W. Liu, and X. Wang, "Gausstr: Foundation model-aligned gaussian transformer for self-supervised 3d spatial understanding," in *CVPR*, 2025.
- [9] X. X. X. H. J. L. J. M. Runsheng Xu, Hao Xiang, "Opv2v: An open benchmark dataset and fusion pipeline for perception with vehicle-to-vehicle communication," in *ICRA*, 2022.
- [10] R. Song, C. Liang, H. Cao, Z. Yan, W. Zimmer, M. Gross, A. Festag, and A. Knoll, "Collaborative semantic occupancy prediction with hybrid feature fusion in connected automated vehicles," in *CVPR*, 2024.
- [11] M. Hu, W. Yin, C. Zhang, Z. Cai, X. Long, H. Chen, K. Wang, G. Yu, C. Shen, and S. Shen, "Metric3d v2: A versatile monocular geometric foundation model for zero-shot metric depth and surface normal estimation," 2024.
- [12] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.
- [13] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik, and A. Kanazawa, "gsplat: An open-source library for gaussian splatting," *Journal of Machine Learning Research*, 2025.
- [14] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A multimodal dataset for autonomous driving," in *CVPR*, 2020.
- [15] A. Vaswani, "Attention is all you need," in *NeurIPS*, 2017.